

Empirical Analysis and Detection of Hallucinations in LLM-Generated Bug Report Summaries

1st Hinduja Nirujan

Electrical and Software Engineering
University of Calgary
Calgary, Alberta, Canada
hinduja.nirujan@ucalgary.ca

2nd Shreyas Patil

Electrical and Software Engineering
University of Calgary
Calgary, Alberta, Canada
shreyas.patil@ucalgary.ca

3rd Abdallah Ayoub

Electrical and Software Engineering
University of Calgary
Calgary, Alberta, Canada
abdallah.ayoub@ucalgary.ca

4th Ahmad Abdel Latif

Electrical and Software Engineering
University of Calgary
Calgary, Alberta, Canada
ahmad.abdellatif@ucalgary.ca

5th Gouri Ginde

Electrical and Software Engineering
University of Calgary
Calgary, Alberta, Canada
gouri.ginde@ucalgary.ca

Abstract—Background: Large Language Models (LLMs) are increasingly used to generate summaries of software bug reports, including sections such as Steps-to-Reproduce (S2R), Actual Behavior (AB), and Expected Behavior (EB). However, these models frequently produce hallucinations that can be convincing but are unsupported by the source bug report. This can mislead developers and reduce trust in automated software maintenance tools. Existing hallucination detection approaches typically evaluate outputs at the full-response level and do not consider the structural properties of technical documents. An initial exploratory study on 80 structured bug report summaries revealed that approximately 47.9% contained missing information, while 12.3% included fabricated content, highlighting the need for systematic hallucination analysis in bug report summarization. **Aims:** In this work, we empirically investigate hallucinations in LLM-generated bug report summaries from a section-aware perspective. **Method:** Using the BugsRepo dataset (multifaceted dataset derived from Mozilla OSS projects), we introduce controlled synthetic hallucination injection to construct a benchmark for training and evaluation. We propose a section-aware hallucination detection approach that jointly predicts whether a summary contains hallucinated content, identifies the affected bug report sections, and classifies hallucination types. **Results:** Experimental results across multiple pretrained language models show that the proposed approach achieves strong performance across all tasks, with the best model obtaining 0.89 report-level Macro-F1, 0.83 section-level Macro-F1, and 0.84 hallucination-type Macro-F1. We further analyze common hallucination patterns and model failure modes to better understand the limitations of current LLM-generated bug report summaries. **Conclusions:** The findings highlight the importance of section-aware hallucination analysis for improving the reliability of LLM-assisted bug report summarization in software maintenance workflows.

[17]. The major hurdle in the software development industry is finding and fixing software issues, including bugs, which can affect functionality and security vulnerabilities that can put the system at risk. This effort is substantial as prior work reports that developers spend, on average, about 50% of their time finding and fixing bugs [11].

The core of bug resolution relies on the bug reports, but their effectiveness relies on the details present in the bug report. Well-structured reports are clearly able to articulate actual behavior(AB), expected behavior(EB), and steps to reproduce (S2R). This structuring minimizes ambiguity and enables developers to resolve issues without much discussion and clarification [35]. With recent advancements in Natural Language Processing (NLP) and Large Language Models (LLMs) showed strong capabilities of in various Software Engineering (SE) tasks such as bug report generation and summarization [4]. The models have demonstrated significant potential in assisting software development and maintenance activities. However, despite their promising performance, LLMs are prone to generating plausible-sounding yet factually incorrect information, commonly referred to as *hallucinations*. This phenomenon poses a significant challenge to the reliability and trustworthiness of LLM outputs, particularly in critical SE tasks where accuracy and faithfulness to the source information are essential. Therefore, detecting and mitigating hallucinations in LLM-generated output (e.g., bug reports) is crucial for ensuring their practical adoption in real-world SE workflows.

Existing hallucination detection methods can be broadly classified into: self-consistency [12], knowledge-graph two-stage detection [16], fine-tuned classifiers, and LLM-as-a-Judge [10]. While these techniques have advanced the field, there has been little research specifically examining hallucinations in the SE domain. Moreover, these methods treat hallucinations as a whole and monolithically, considering them as a single class of errors. This perspective fails to capture

I. INTRODUCTION

Software maintenance is a major expense in the software development process, costing billions of dollars every year [15]. For example, the Consortium for Information & Software Quality estimated that poor software quality cost the U.S. at least \$2.41 trillion in 2022, with accumulated technical debt

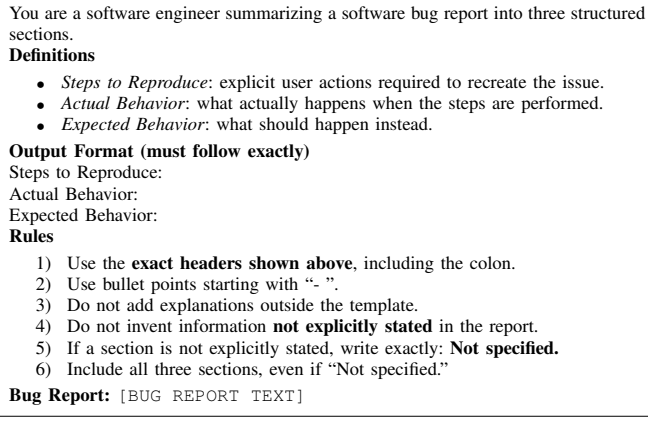


Fig. 1. Prompt template used to generate structured bug report summaries.

the diversity of factual errors and overlooks how hallucination patterns can vary across different semantic sections of a bug report. As a result, these methods do not account for the distinct roles and contextual behaviors of sections. Understanding hallucination patterns is crucial, as different types of hallucinations may require different detection and mitigation strategies depending on the semantic context and functional role of each section in the bug report.

Motivation. To better understand the behavior of hallucinations in LLM-generated bug report summaries, we conducted an exploratory analysis on a subset of the BugsRepo dataset [3], a multifaceted curated dataset derived from Mozilla projects of bug reports organized into semantic fields such as S2R, AB, and EB. Specifically, we randomly selected 80 bug reports from the BugsRepo unstructured dataset, based on their structural characteristics and length. Next, we prompted Llama 3.2 using a prompt that contains explicit definitions and rules as shown in Fig.1, with the temperature set to 0.0 to ensure deterministic outputs. The summaries were generated under two compression settings according to the report length distribution, where low compression allowed up to 180 tokens and high compression allowed 90 tokens. Each summary was then independently and anonymously annotated by three authors using three labels: supported, incomplete, and fabricated. The agreement between the annotators was measured using Cohen’s Kappa [5], giving a high agreement (Kappa = 0.85). After resolving disagreements through discussion among the annotators, the final labels were established.

Figure 2 presents the results of this analysis. From the figure, we observe that hallucinations are prevalent in LLM-generated bug report summaries, with over 60% of summary segments containing either incomplete or fabricated information. Further, hallucination rates vary across semantic sections of bug reports, suggesting that hallucination behavior is not uniform and may depend on the contextual role of each section.

To address this gap, we propose a section-aware hallucination detection approach for structured bug report summaries. Using BugsRepo [3] subset of structured reports as the source corpus, we constructed a synthetic hallucination dataset by injecting controlled inconsistencies into specific semantic sec-

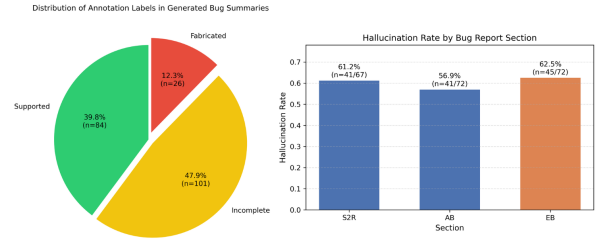


Fig. 2. Results of the exploratory analysis of hallucinations in LLM-generated bug report summaries.

tions, then fine-tuned language models to detect hallucinated sections and classify hallucination types. More specifically, our study answers the following three research questions (RQs) that progressively address the key components required for this approach:

- **RQ1:** *How does hallucination sensitivity vary across different sections of structured bug reports during natural language generation?* Before defining our methodology, we analyze how transformer models distribute attention across different sections. These insights provide a foundation for our synthetic dataset distribution and help validate the empirical behaviors observed during zero-shot prompting, increasing confidence in our results.
- **RQ2:** *How can structured bug reports be reliably converted into high-quality unstructured reports for training hallucination detection models?* Prior work typically evaluates structured-to-unstructured report conversions using semantic similarity metrics like cosine similarity [3]. However, our experiments show that these metrics fail to capture the core technical essence of structured reports, highlighting the need for a more reliable verification pipeline.
- **RQ3:** *How effectively can a multi-task learning model detect hallucinations, identify affected sections, and classify hallucination types in bug report summaries?* An effective hallucination guardrail must first detect if an error exists before identifying the affected sections and hallucination types. This hierarchical dependency requires a multi-task learning approach, enabling the model to learn that section localization and type classification are conditional upon primary error detection.

Through these RQs, our goal is to empirically study hallucinations in LLM-generated bug report summaries, analyze section-level vulnerability, construct a filtered benchmark of structured-to-unstructured conversions with controlled hallucination injection, and evaluate a multitask detector for report-, section-, and type-level hallucination detection. Our specific contributions are as follows:

- 1) We introduce a novel section-aware approach for analyzing hallucinations in LLM-generated bug report summaries by explicitly modeling the semantic roles of S2R, AB, and EB for jointly detecting hallucinated summaries, identifying affected sections, and classifying hallucination types.

- 2) We develop a synthetic dataset construction pipeline that converts structured bug reports into unstructured generated bug reports and creates hallucination-injected structured summaries to enable systematic section-level hallucination experiments. We make this benchmarking dataset publicly available for researchers to use and experiment ¹.
- 3) We provide empirical insights into hallucination behavior across bug report sections, highlighting patterns that can inform the development of more reliable LLM-assisted bug reporting tools.

Paper Organization. Section II reviews the related work. Section III describes the dataset construction process, model setup, and experimental design. The Section IV explains the analysis and answers to each RQ. Section V presents the discussion and outlines directions for future work. Section VI discusses the threats to validity, and Section VII concludes the paper.

II. RELATED WORK

In this section, we will present work related to bug report summarization, and hallucination detection in SE artifacts.

Bug Report Summarization. Automatic bug report summarization aims to condense lengthy and noisy issue discussions into concise representations that help developers understand and resolve bugs efficiently. Early work emphasized content-aware and noise-aware unsupervised summarization methods for bug reports. Mani et al. proposed AUSUM, an unsupervised approach that reduce noise sentences from bug reports by considering sentence structure and topic relevance[24]. Lotufo et al. modeled the 'hurried' bug report reading process and ranked sentences according to discussed topics, evaluation and report title signals [22]. Jiang et al. proposed PRST, a PageRank-based summarization technique that considers duplicate bug reports, showing that duplicate discussions can provide useful evidence for identifying important report content [14]. More recent approaches adopt pre-trained representations and contrastive objectives to improve bug report summarization. RepresentThemAll learns a universal bug report representation for several downstream tasks, including summarization[8].KSCLP adds knowledge-specific pre-training before Seq2Seq summary generation [30], and SumLLaMA adapts LLMs using component-level contrastive pre-training and LoRA-based fine-tuning [36].

While these approaches improve overall summary generation, recent studies have started to focus on structured bug report summaries. Mukhtar et al. proposed ClaSum, a multidocument summarization approach that combines classification and summarization to cover key bug attributes such as bug description, reproduction steps, environment, and solutions [27]. Acharya and Ginde studied whether instruction-tuned LLMs can transform casual, unstructured bug reports into structured reports containing fields such as S2R, EB, and

AB [4]. BRMDS further formulates bug report summarization as a multi-dimensional generation task, producing summaries across environment, actual behavior, expected behavior, bug category, and solution suggestions using PEFT-tuned LLMs [37]. However, the existing work primarily evaluates summary informativeness, semantic similarity, and overall generation quality. They do not explicitly assess whether each generated section is factually grounded in the source bug report. This motivates our study of hallucination detection in structured bug report summaries, especially at the section level.

Hallucination Detection in SE Artifacts. Hallucinations in LLMs refer to outputs that are factually incorrect, unsupported by context, or logically inconsistent. This problem has been widely studied in open-domain NLP tasks such as question answering, dialogue generation, and text summarization [25], [13], [2]. Recent work includes self-consistency, model-based evaluation, and taxonomy-driven analyses. SelfCheckGPT uses self-consistency by sampling multiple responses to the same prompt and detecting hallucinations from factual inconsistencies across those samples [12]. SelfCheckEval extends this with black-box, semantic and contextual consistency checks [26], while taxonomy-driven studies categorize different hallucination types [29]. However, hallucination detection in SE requires additional attention because LLM outputs may become artifacts that developers inspect, reuse, or execute.

Recent SE research shows that hallucinations occur across both executable and natural-language software artifacts. In code generation, hallucinations may appear as syntactically plausible but incorrect code, fabricated APIs, nonexistent dependencies, or logic that violates the intended functionality. Prior work characterizes code hallucinations through factuality, faithfulness, and compatibility, covering API errors, requirement deviations, and dependency or environment assumptions [9], [7], [21], [38]. Package hallucination studies show that LLMs can generate nonexistent package names, creating software supply-chain risks [18], [33]. Recent research also studies hallucinations in code-to-natural-language artifacts. ETF [23] detects hallucinations in code summaries by tracing generated entities back to source code. Liu et al. show that generated code reviews and commit messages can contain input inconsistencies, logic inconsistencies, and intent deviations [20]. Similarly, HalluJudge detects whether generated review comments are grounded in the corresponding code diff without the reference comments [32].

These studies show that SE hallucination detection requires a grounding generated text in technical evidence such as source code, code diffs, APIs, etc. Handling bug reports can be a challenge as they combine natural language with structured technical evidence. The CHIME approach [31] studies inaccuracies in ChatGPT responses to bug-report questions and shows that LLMs struggle with stack traces, metadata, and context distributed across issue-report text. This is relevant to our setting because bug report hallucination detection must account for technical artifacts embedded in natural-language reports. These studies highlight the unique challenges of the SE context, where factual grounding depends on structured

¹https://anonymous.4open.science/r/Anon_Bug_Report_Summarization-1477

technical elements rather than purely natural language. Unlike prior work that focuses on whole-response hallucination detection or code-oriented artifacts, our work specifically examines whether semantic sections such as S2R, AB, and EB are factually grounded in the source bug report.

Our work investigates section-level hallucination in structured bug report summaries, where each semantic section must be grounded in the corresponding source report. This approach enables a more fine-grained evaluation of hallucination detection methods for SE artifacts.

III. METHODOLOGY

The overall methodology of the proposed section-aware hallucination detection approach is shown in Fig.3.

Dataset. BugsRepo [3] is a curated dataset derived from Mozilla Bugzilla projects, containing bug reports along with metadata, comments, and contributor information. Relevant to our work, BugsRepo includes a structured subset of 10,351 well-formed reports with S2R, AB, and EB fields. Since our study focuses on section-aware hallucination in structured bug report summaries, we use this structured subset. After preprocessing and removing unusable records, our experiments use 10,304 structured bug reports created between January 2021 and November 2024 for section attention analysis, structured-to-unstructured report conversion, and synthetic hallucination injection components.

A. Section Attention Analysis

Section Extractor. In the BugsRepo dataset, bug reports are provided in a semi-structured format containing three primary components: S2R, AB, and EB. Despite this structural organization, the reports frequently include extraneous content such as human commentary, stack traces, and references to GitHub pull requests. Incorporating such artifacts into section-level analysis can introduce noise and bias the distribution of attention, as irrelevant tokens may dilute or overshadow semantically meaningful information within each section. To address this challenge, we propose a deterministic, rule-based extraction approach that isolates the core semantic components of bug reports. The study leverages lexical pattern matching and structural heuristics to accurately identify and segment relevant sections while filtering out non-informative content. It is specifically designed to be robust against formatting inconsistencies and the noisy, heterogeneous nature of real-world bug reports, thereby preserving the integrity of section-level representations for downstream analysis.

Attention Analysis. After isolating individual S2R, AB, and EB sections, our objective is to empirically evaluate how transformer architectures distribute contextual importance across these distinct semantic regions. To achieve this, we analyze intra-section and inter-section attention dynamics, leveraging multi-head self-attention weights to quantify information routing and cross-field dependencies within the bug reports. Our entire process for analyzing and aggregating multi-model attention distributions is outlined in algorithm 1.

To ensure a broad coverage in terms of different text-processing models, we consider multiple prominent pretrained transformers, such as BERT, RoBERTa, DistilBERT, and even domain-adapted models, like SciBERT, and DeBERTa, which have been shown to be successful in analyzing contextual relationships within software artifacts [34], [3]. A key feature of the mentioned models is their multi-head self-attention architecture, which allows us to measure token-to-token interactions. For each bug report, we create a single sequence of tokens that explicitly encodes the extracted section borders:

$$\begin{aligned} T &= [\text{CLS}] [\text{S2R}] T_{s2r} [\text{AR}] T_{ar} [\text{ER}] T_{er} [\text{SEP}], \\ \mathcal{I} &= \{I_{s2r}, I_{ar}, I_{er}\}. \end{aligned} \quad (1)$$

where T_{s2r} , T_{ar} , and T_{er} denote the tokenized representations of the S2R, AB, and EB sections, respectively. The index sets in $\mathcal{I}$ enable section-level analysis by isolating tokens belonging to each section.

These index mappings allow precise isolation of tokens belonging to each section. Given an input sequence of length N , transformer models compute attention using multi-head self-attention across L layers and H attention heads. For each layer l , the attention tensor is defined as:

$$\begin{aligned} A^{(l)} &\in \mathbb{R}^{H \times N \times N}, \\ A &= \frac{1}{L} \sum_{l=1}^L \left(\frac{1}{H} \sum_{h=1}^H A_h^{(l)} \right). \end{aligned} \quad (2)$$

where H is the number of attention heads and N is the number of tokens. Each head captures token-to-token interactions, where $A_h^{(l)}[i, j]$ represents the attention weight from token i to token j for head h in layer l . The resulting matrix $A \in \mathbb{R}^{N \times N}$ encodes the average attention distribution across all token pairs. To quantify intra-section interactions, we compute the average attention between tokens within the same section. For a section $S \in \{s2r, ar, er\}$, the attention score is defined as:

$$\begin{aligned} \text{Attn}(S) &= \frac{1}{|I_S|^2} \sum_{i \in I_S} \sum_{j \in I_S} A[i, j], \\ \text{Attn}(S \rightarrow T) &= \frac{1}{|I_S| \cdot |I_T|} \sum_{i \in I_S} \sum_{j \in I_T} A[i, j]. \end{aligned} \quad (3)$$

This metric captures the internal coherence of each section. To analyze inter-section dependencies, we compute directional attention scores between different sections. For two distinct sections S and T , the cross-section attention score is defined using $\text{Attn}(S \rightarrow T)$. This formulation captures the extent to which tokens in section S attend to tokens in section T . The overall workflow for calculating these metrics across the base architectures is formalized in Algorithm 1.

B. Synthetic Dataset Construction

As there are no available datasets for hallucinations in section-aware bug report summaries, we constructed a benchmark dataset from the structured subset of the BugsRepo dataset.

Unstructured Report Conversion. The structured reports

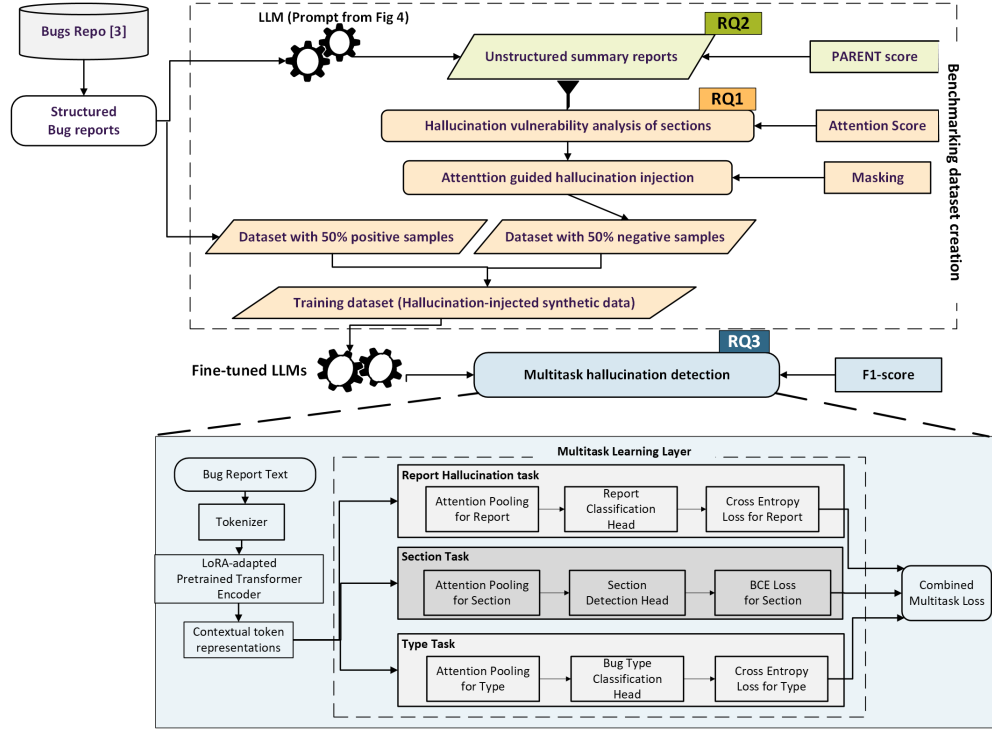


Fig. 3. Our proposed section-aware hallucination detection approach and study design

are converted into unstructured reports using the Mistral-7B model using a refined prompt 4. Since automatically generated reports may contain hallucinated or missing information, it is necessary to filter unreliable generations before constructing the training dataset. To evaluate this, we employ the PARENT (Precision and Recall of Entailed N-grams from the Table) metric [6]. PARENT evaluates table-to-text generation by measuring the agreement between the generated text G , the reference text R , and the source table T . The metric computes entailed precision and entailed recall over n-grams, rewarding generated tokens that either appear in the reference text or can be inferred from the source table. The overall PARENT score is defined as the harmonic mean of entailed precision (E_p) and entailed recall (E_r):

$$\text{PARENT}(G, R, T) = \frac{2 \cdot E_p \cdot E_r}{E_p + E_r}. \quad (4)$$

However, a complete reference description for each structured bug report is not available in the dataset. It contains a summary field that provides a brief summary of the reports. Therefore, relying on reference-based recall may underestimate the correctness of generated content that faithfully reflects the structured report. To address this limitation, we introduce a simplified variant of the metric that prioritizes alignment with the source table. Specifically, we compute a modified score defined as:

$$\text{PARENT}_{\text{table}} = 0.5 \cdot E_p + 0.5 \cdot E_r(T). \quad (5)$$

where E_p denotes the *entailed precision* from the original PARENT metric. In contrast, $E_r(T)$ corresponds only to the

Prompt used for structured to unstructured report conversion Rewrite the following structured bug report as one coherent paragraph using the original wording as much as possible. **Rules:**

- Preserve the original meaning exactly.
- Do not invent, infer, summarize, or generalize.
- Do not drop important details.
- Keep all technical identifiers and quoted strings unchanged.
- Keep the distinction between expected behavior and actual behavior explicit.
- Use a single paragraph only.
- Make it readable, but stay as close as possible to the source wording.

Structured bug report:

[BUG REPORT TEXT]

Fig. 4. Prompt template used to convert structured bug reports to unstructured reports.

recall from the source table, measuring how much of the structured report content is covered in the generated text, unlike the original PARENT formulation, which combines recall from both the reference text and the source table. The reasons and empirical evaluation of this modification are discussed further in Section IV-B. Following the filtering procedure described in Section IV-B, low-quality conversions were removed to ensure reliable structured-to-text alignment. After filtering, 7,684 high-quality unstructured bug reports were retained from the original 10,304 reports. From this filtered dataset, approximately 50% of the samples (3,800 reports) were selected for synthetic hallucination injection, while the remaining reports were preserved as non-hallucinated instances.

Synthetic Hallucination Injection The annotated datasets for studying hallucinations and factual inconsistencies are limited. Creating this data manually is both time-consuming and prone to human error. Prior work has used synthetic

Algorithm 1: Section-wise and Cross-section Attention Analysis

Input: Bug report dataset D with sections S2R, AR, and ER

Output: Section-wise and cross-section attention scores

```
1  $M \leftarrow$ 
   {BERT, RoBERTa, SciBERT, DeBERTa, DistilBERT};
2 foreach bug report  $d \in D$  do
3   Extract and clean sections  $s2r$ ,  $ar$ , and  $er$ ;
4   Construct token sequence  $T \leftarrow$ 
     [CLS], [S2R],  $T_{s2r}$ , [AR],  $T_{ar}$ , [ER],  $T_{er}$ , [SEP];
5   Identify section index sets  $I_{s2r}$ ,  $I_{ar}$ , and  $I_{er}$ ;
6   foreach model  $m \in M$  do
7     Obtain attention tensors  $A^{(l)}$  for all layers;
8     Compute the averaged attention matrix
        $A \leftarrow \frac{1}{L} \sum_{l=1}^L \left( \frac{1}{H} \sum_{h=1}^H A_h^{(l)} \right)$ ;
9     foreach section  $S \in \{s2r, ar, er\}$  do
10      Compute intra-section attention
         $\text{Attn}(S) \leftarrow \frac{1}{|I_S|^2} \sum_{i \in I_S} \sum_{j \in I_S} A[i, j]$ ;
11      foreach ordered pair  $(S, T)$  such that
         $S, T \in \{s2r, ar, er\}$  and  $S \neq T$  do
12      Compute cross-section attention  $\text{Attn}(S \rightarrow$ 
         $T) \leftarrow \frac{1}{|I_S||I_T|} \sum_{i \in I_S} \sum_{j \in I_T} A[i, j]$ ;
13      Store attention scores for model  $m$ ;
14 return Aggregated attention dataset;
```

perturbations to simulate hallucinated outputs and augment hallucination detection data [25], [1]. Based on this, we design a controlled hallucination injection pipeline designed to systematically perturb structured bug report fields using an LLM. The objective is to generate semantically altered yet syntactically valid variants under strict constraints.

The hallucination injection process begins by iterating over each bug report in the dataset (Lines 1–2), where only those marked for hallucination are selected for further processing. For each such instance, the corresponding field and hallucination type are identified (Line 3). The pipeline then enters an iterative generation phase (Lines 4–8), where the LLM produces candidate outputs conditioned on the selected prompt. Each generated output is cleaned and validated against strict constraints specific to the hallucination type. The iteration terminates early if a valid transformation is obtained, ensuring efficiency while maintaining correctness. In cases where none of the generated outputs satisfy the validation criteria, the final attempt is retained and explicitly marked as a failure (Lines 9–11). Subsequently, the resulting output, along with metadata such as success status and number of attempts, is recorded (Line 12). To ensure robustness and fault tolerance, the pipeline periodically saves intermediate results, enabling recovery and seamless continuation in case of interruptions (Line 14).

Algorithm 2: Hallucination Injection Pipeline

Input: Dataset D , maximum number of retries K

Output: Injected hallucination examples with success flags and attempt counts

```
1 foreach report  $r \in D$  do
2   if  $r$  is marked for hallucination then
3     Select field  $f$  and hallucination type  $h$ ;
4     for  $k \leftarrow 1$  to  $K$  do
5       Generate  $f'_k \leftarrow \text{LLM}(f, p_h)$ ;
6       Clean output  $f'_k$ ;
7       if Validate( $f, f'_k, h$ ) then
8         Accept  $f' \leftarrow f'_k$ ;
9         break;
10    if no valid output is found then
11      Assign  $f' \leftarrow f'_K$  and mark as failed;
12    Store  $f'$ , success flag, and attempts;
```

C. Section-Aware Hallucination Detection

In the Section Aware detection, we evaluate multiple frontier LLMs; the previous approaches have shown that fine-tuned LLMs have higher performance [4] in terms of bug report summarization. From the synthetic data, we split the data for fine-tuning and testing the fine-tuned LLM. For each instance, we evaluate the LLM detection with metrics like accuracy, precision, F1-score, and recall.

Multitask Hallucination Detector Existing hallucination detection approaches [12], [16], [10], [29], [26] typically frame the problem as a binary classification task that predicts whether a generated text contains hallucinated content. While this formulation is adequate for general text generation tasks, it is insufficient for structured bug reports. In bug reports, hallucinations may occur within specific sections and may be evident in different forms, such as additions, omissions, or reordering of structured information. Therefore, detecting the presence of hallucination alone does not provide sufficient insight into the location or type of the error.

To address this limitation, we propose a section-aware hallucination detection approach that simultaneously predicts (i) whether a report contains hallucinated content, (ii) which section of the bug report is affected, and (iii) the type of hallucination. The model learns three related tasks: a report-level hallucination classification task, a section-level localization task that identifies hallucinations in the S2R, AB, or EB sections, and a hallucination type classification task that distinguishes between *addition*, *removal*, and *reordering* errors based on the results of the initial exploratory analysis explained in I. As described in Figure 3, the model is built upon a pretrained transformer encoder that generates contextual token representations for the input bug report. Task-specific representations are obtained using attention-based pooling layers, which aggregate contextual token representations relevant to each task. These pooled representations are then passed

to three prediction heads: a binary classifier for report-level hallucination detection, a multi-label classifier for section-level localization, and a multi-class classifier for hallucination type prediction. The model is trained using a multitask objective that combines the losses from the three tasks:

$$\mathcal{L} = \mathcal{L}_{report} + \lambda_s \mathcal{L}_{section} + \lambda_t \mathcal{L}_{type}. \quad (6)$$

where λ_s and λ_t control the relative importance of the auxiliary tasks. To reduce computational cost, parameter-efficient fine-tuning (PEFT) is applied using Low-Rank Adaptation (LoRA), which updates small trainable adapter parameters in the transformer attention layers, together with the task-specific pooling layers and prediction heads.

IV. RESULTS

After detailing a section-aware hallucination detection approach, we describe the methodology and present the results for each research question in this section.

A. Answering RQ1: Hallucination sensitivity

To analyze hallucination sensitivity across different sections of structured bug reports, we examine the average attention distribution across different sections of bug reports, namely S2R, AB, and EB, using multiple transformer models. As

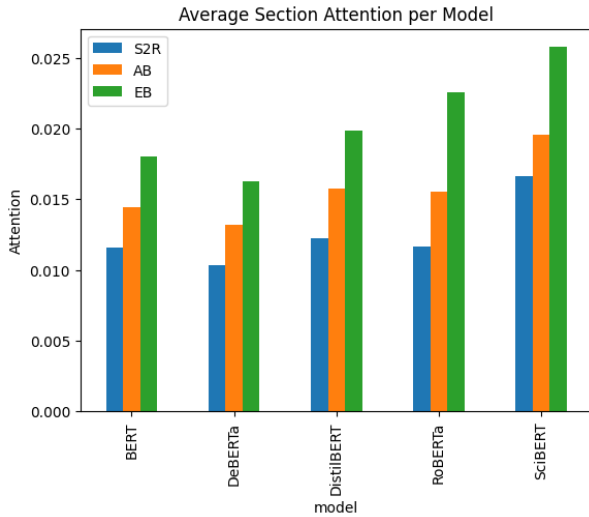


Fig. 5. Average section-wise attention across transformer models.

shown in Figure 5, a consistent trend is observed across all models: EB receives the highest attention. AB receives moderate attention, and S2R receives the lowest attention. Attention reflects the relative importance assigned to different sections during representation learning. The lower attention assigned to S2R indicates that models under-utilize procedural and sequential information, making it more susceptible to hallucination during generation. In contrast, EB sections, which are typically concise and declarative, receive higher attention and are less prone to hallucination. Guided by these observations, we construct a hallucination-sensitive dataset (half of the original dataset) by prioritizing sections with lower attention. The overall distribution is, S2R: 1600 samples, EB:

TABLE I
DISTRIBUTION OF INJECTED HALLUCINATION SAMPLES BY BUG REPORT SECTION AND HALLUCINATION TYPE.

Section	Add	Remove	Reorder
EB	600	600	–
AB	500	500	–
S2R	584	583	433

1200 samples, and AB: 1000 samples. Each section is further divided into hallucination types, as shown in the Table I.

Answer to RQ1: The results indicate a strong correlation between attention allocation and hallucination sensitivity. Sections with lower attention, particularly S2R, exhibit higher susceptibility to hallucination. This insight directly informs the dataset design, ensuring that evaluation focuses on structurally challenging and under-attended components of bug reports.

B. Answering RQ2: Converting structured bug reports into high-quality unstructured reports reliably for training hallucination detection models

Evaluation Metric Justification. Evaluating structured-to-text generation is challenging because traditional lexical similarity metrics such as BLEU [28] and ROUGE [19] may not accurately reflect the similarity of the generated reports with the structured bug reports and penalize paraphrased outputs. In our setting, the requirements is not only lexical or semantic similarity but preserving of the information in the sections of structured bug reports. To address this limitation, we employ the PARENT metric and a variant of the PARENT metric $PARENT_{table}$, which evaluates factual alignment between structured input data and generated text. To validate the suitability of PARENT for this task, Pearson correlation coefficients were computed across all generated 10,304 reports. Table II presents the correlation between different evaluation metrics and PARENT scores. The results indicate moderate correlations between BLEU, ROUGE, and PARENT (0.61–0.67), suggesting that lexical overlap captures only part of the generation quality. In addition, BERTScore-F1 shows lower correlations with $PARENT_{bug}$ ($r = 0.504$) and $PARENT_{table}$ ($r = 0.471$), while correlating more strongly with ROUGE-1 ($r = 0.740$) and ROUGE-L ($r = 0.725$). This indicates that BERTScore captures contextual similarity between texts but does not fully substitute for source-field preservation. In contrast, source recall shows the strongest correlation with PARENT (0.73–0.79), indicating that PARENT more closely reflects the preservation of structured bug information during conversion. Therefore, we use PARENT as the primary metric for dataset curation. We further inspected row-level cases where BERTScore-F1 was high but $PARENT_{table}$ was low. These cases show that generated reports can remain semantically similar to the source while omitting or weakening source-field information from S2R, AB, or EB. Therefore,

TABLE II
CORRELATION OF DIFFERENT EVALUATION METRICS WITH PARENT
AND PARENT_{table} SCORES.

Metric	PARENT Corr.	PARENT _{table} Corr.
BLEU	0.654	0.660
ROUGE-1	0.675	0.643
ROUGE-L	0.640	0.615
Cosine TF-IDF	0.645	0.716
BERTScore-F1	0.504	0.471
Reference Recall	0.251	0.222
Source Recall	0.787	0.735

BERTScore alone is insufficient for filtering structured-to-text conversions. The inspected examples are included in ²

Generation Configuration. After establishing PARENT as the primary evaluation metric, generation parameters were tuned to maximize the preservation of structured bug information in the generated reports. The PARENT_{table} score was used as the optimization metric. Three prompt formulations were evaluated: a *Baseline Prompt* adapted from prior work [4] which rewrites the report into a paragraph, a *Natural Narrative Prompt* designed to produce fluent human-like descriptions, and a more constrained, and *Strict Prompt* that emphasizes strict preservation of the original wording and structured information. We use these prompts to compare natural rewriting against stricter source-faithful conversion. Among these, the *Strict Prompt* produced the more consistent PARENT_{table} scores (mean = 0.564, median = 0.575) and was selected for dataset generation. Sampling temperature was varied between 0.0 and 1.0 to analyze the effect of generative variability on factual consistency, with lower temperatures yielding slightly higher PARENT_{table} scores; a temperature of 0.2 was therefore adopted. Finally, the generation length was dynamically scaled according to the source report length to ensure sufficient coverage of structured content while maintaining concise outputs.

Conversion Quality and Dataset Filtering. After generating the narrative bug reports using the selected configuration (Strict prompt, 0.2 temperature, and dynamic generation length), the quality of the conversions was analyzed using the PARENT_{table} metric. Figure 6(a) shows the distribution of PARENT_{table} scores across the generated dataset, indicating that around 75% reports achieve moderate to high scores and therefore preserve the structured bug information effectively. However, a portion of the samples exhibit lower scores, corresponding to cases where important structured fields are partially omitted or insufficiently expressed in the generated text.

To ensure dataset reliability, low-quality conversions were removed using a filtering threshold of PARENT_{table} ≥ 0.50. As shown in Figure 6(b), this threshold retained 7,768 reports (75.39%) and filtered out 2,536 lower-scoring reports (24.61%). The selected threshold lies near the beginning of the steep retention-loss region, providing a more conservative

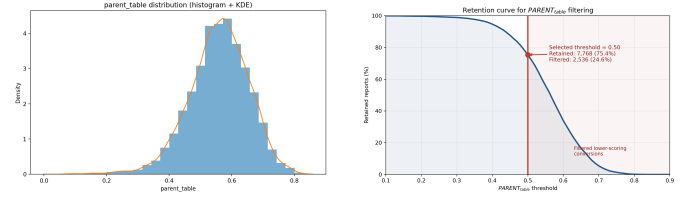


Fig. 6. Quality analysis of generated bug report descriptions. (a) Distribution of PARENT_{table} scores across generated reports, showing that most conversions preserve structured bug information. (b) Filtering strategy based on the score distribution, where low-scoring outliers are removed to ensure dataset reliability.

filter than a relaxed threshold. This results in a filtered dataset that maintains strong semantic consistency with the original structured bug reports.

Answer to RQ2: The evaluation confirms that PARENT_{table} is a suitable metric for structured-to-text bug report conversion. Using PARENT_{table} to guide prompt selection, temperature tuning, and filtering, the generation pipeline produces unstructured reports that preserve the structured information from the original bug reports.

C. Answering RQ3: Multi-task learning model to detect hallucinations, identify affected sections, and classify hallucination types.

To evaluate the effectiveness of the proposed section-aware hallucination detection approach, we fine-tuned the models using the constructed synthetic dataset. The dataset was divided into training, validation, and test sets using a 70%, 10%, 20% split for fine-tuning, model fitting, and final evaluation, respectively. Experiments were conducted using four pretrained language models with different parameter scales: Mini-LLaMA-1B, Ministral-3B, Mistral-7B, and LLaMA-8B, which were selected to evaluate the impact of model scale on section-aware hallucination detection performance under practical computational constraints. The multitask model jointly predicts three tasks: (i) report-level hallucination detection, (ii) section-level hallucination localization, and (iii) hallucination type classification. The performance is evaluated using accuracy and F1 metrics with task-specific confusion matrices.

Model Performance. Table III summarizes the performance of the proposed section-aware hallucination detection approach across all fine-tuned models and prompting baselines. The results demonstrate that compared to zero-shot, few-shot, and CoT prompting, all fine-tuned models achieve strong performance (F1-score ≥ 0.8) across the three tasks, including report-level hallucination detection, section-level hallucination localization, and hallucination type classification.

For report-level hallucination detection, Ministral-3B achieved the best performance, with an accuracy of 0.898 and a macro-F1 score of 0.891. Mistral-7B and LLaMA-8B also achieved comparable performance, with macro-F1 scores of 0.881 and 0.885, respectively. Mini-LLaMA-1.1B achieved lower performance, reflecting the limited representational capacity of smaller models. For section-level hallucination localization,

²https://anonymous.4open.science/r/Anon_Bug_Report_Summarization-1477/unstructured_conversion/Data/final_conversion/metric_comparison/metric_comparison_bert_vs_parent_examples.md

TABLE III
OVERALL PERFORMANCE COMPARISON OF PROMPTING AND FINE-TUNED SECTION-AWARE HALLUCINATION DETECTION MODELS.

Model	Approach	Prompt	Report Acc.	Report Macro-F1	Section Micro-F1	Section Macro-F1	Type Macro-F1
LLaMA-8B	Prompting	Zero-shot	0.493	0.480	0.244	0.230	0.277
	Prompting	Few-shot	0.550	0.550	0.398	0.407	0.295
	Prompting	CoT	0.422	0.314	0.327	0.310	0.182
Mistral-7B	Prompting	Zero-shot	0.578	0.427	0.073	0.072	0.218
	Prompting	Few-shot	0.418	0.308	0.281	0.198	0.193
	Prompting	CoT	0.461	0.457	0.231	0.168	0.205
Ministral-3B	Prompting	Zero-shot	0.683	0.679	0.212	0.212	0.484
	Prompting	Few-shot	0.662	0.661	0.299	0.270	0.454
	Prompting	CoT	0.640	0.639	0.234	0.224	0.477
Mini-LLaMA	Fine-tuned	N/A	0.832	0.816	0.716	0.723	0.662
Ministral-3B	Fine-tuned	N/A	0.898	0.891	0.846	0.829	0.836
Mistral-7B	Fine-tuned	N/A	0.889	0.881	0.832	0.809	0.817
LLaMA-8B	Fine-tuned	N/A	0.892	0.885	0.839	0.820	0.802

TABLE IV
DETAILED PERFORMANCE OF THE BEST-PERFORMING MODEL, MINISTRAL-3B: (A) SECTION LOCALIZATION AND (B) HALLUCINATION TYPE CLASSIFICATION.

(a) Section localization			
Section	Prec.	Rec.	F1
S2R	0.935	0.851	0.891
AB	0.989	0.656	0.789
EB	1.000	0.675	0.806
(b) Type classification			
Type	Prec.	Rec.	F1
Add	1.000	0.954	0.976
Remove	0.831	0.383	0.525
Reorder	0.963	0.888	0.924

Ministral-3B again obtained the highest performance, with a micro-F1 score of 0.846 and a macro-F1 score of 0.829. Mistral-7B and LLaMA-8B followed closely, while Mini-LLaMA-1.1B showed a larger performance gap. The improvement over Mini-LLaMA suggests that richer contextual representations help the model better capture structural inconsistencies across sections.

For hallucination type classification, Ministral-3B achieved the best macro-F1 score of 0.836, outperforming Mistral-7B and LLaMA-8B. This indicates that increasing model size alone does not necessarily lead to better hallucination-type classification. Overall, the results indicate that the proposed section-aware learning approach can effectively detect hallucinated bug report summaries while simultaneously identifying the affected sections and the type of hallucination. Although the 3B, 7B, and 8B models achieve broadly comparable performance, Ministral-3B provides the strongest overall results while requiring fewer computational resources than the larger models.

Section-Level Hallucination Localization. To further analyze the model in localizing hallucinations, we evaluated section-level detection performance for the best-performing model,

Ministral-3B. Table IV(a) reports the precision, recall, and F1-scores for detecting hallucinations in each bug report section. The model achieved the highest performance for the S2R section with an F1 of 0.891. Performance was slightly lower for the AB and EB sections, which obtained F1-scores of 0.789 and 0.806, respectively. Across all sections, the model achieved high precision, especially for AB and EB, but recall was lower for these sections. This suggests that the model is conservative when assigning hallucinations to AB and EB while it predicts these sections, it is usually correct, but it misses some hallucinations in those sections. This difference may be attributed to the more structured nature of reproduction steps, which provide clearer contextual insights for identifying hallucinated content.

Hallucination Type Classification. To evaluate whether the model can distinguish different hallucination patterns, we analyze the hallucination type classification performance of the best-performing model, Ministral-3B. As shown in Table IV(b), the model achieves strong performance for addition and reordering hallucinations, achieving high precision and recall. In contrast, removal-type hallucinations are more difficult to detect, resulting in lower recall. This is expected, as removal errors correspond to missing information rather than explicit textual additions, making them harder to identify. Despite this challenge, Ministral-3B improves removal detection compared with the larger models and achieves strong overall performance across hallucination types.

Qualitative Error Analysis. To better understand the limitations of the proposed detector, we analyzed all incorrect predictions (163 cases) produced by the best-performing model, Ministral-3B, from the 1,354 synthetic test cases. The error set was dominated by removal hallucinations, particularly in declarative sections with AB-removal and EB-removal accounted for 42 and 44 incorrect cases, respectively. We manually coded each incorrect prediction using explanation labels that capture why the prediction failed, including *missing or implicit information*, *procedural confusion*, *section-boundary*

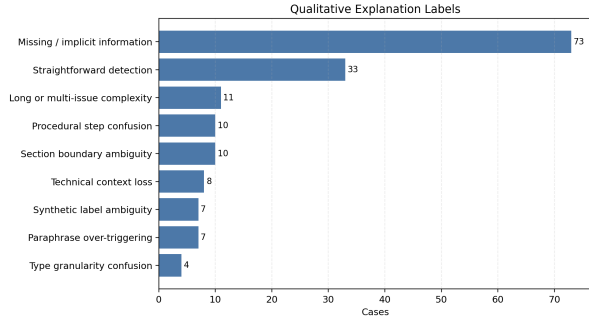


Fig. 7. Frequencies of manual explanation labels from the qualitative error analysis.

ambiguity, *technical context loss*, *paraphrase over-triggering*, *long/multi-issue complexity*, and *synthetic label ambiguity*. To support reliability, 30 cases were independently annotated by three authors, while the remaining 133 cases were distributed. The overlap subset was stratified by ground-truth section and hallucination type. The Fleiss’ kappa was 0.200, and all conflicting cases were resolved through discussion. Figure 7 summarizes the explanation labels distribution. The most frequent explanation was *missing or implicit information hard to notice*, accounting for 73 of 163 cases (44.8%), followed by *straightforward detection failure* with 20.2%, while the remaining categories each accounted for less than 7% of the errors. The dominant failure mode was *missed omitted or implicit information*, appearing in 60.7% of removal cases, 54.6% of report-level false negatives, and 50.5% of high-confidence wrong predictions. The pattern was more common in EB and AB than in S2R, indicating that removal hallucinations are harder because the model must detect absent source information rather than visible unsupported text. False positives were more heterogeneous, often reflecting *paraphrase over-triggering*, *straightforward detection failures*, or *borderline section ambiguity*. Overall, the qualitative analysis shows that the detector’s main limitation is not general hallucination recognition, but detecting omitted or implicitly preserved information, especially in removal hallucinations.

Answer to RQ3: The proposed section-aware approach effectively detects hallucinated bug report summaries while identifying the affected sections and hallucination types. Ministral-3B achieved the best overall performance, showing that strong detection can be obtained with a resource-efficient mid-scale model. Error analysis shows that removal hallucinations, especially in AB and EB sections, remain the most challenging case. The results show that multitask learning enables more detailed and interpretable hallucination detection than traditional binary approaches.

V. DISCUSSION AND FUTURE WORK

The findings of this study highlight the importance of incorporating document structure when analyzing hallucinations in LLM-generated bug report summaries. The exploratory analysis demonstrated that hallucination patterns vary across

semantic sections, suggesting that different contexts influence the summary generation. The multitask learning approach shows that jointly modeling report-level hallucination detection, section identification, and hallucination type classification can provide richer signals for detecting factual inconsistencies. In our experiments, the best-performing model achieved a report-level accuracy of 0.892, while obtaining a section-level micro-F1 of 0.839, demonstrating the effectiveness of the proposed section-aware detection approach.

Despite these promising results, several directions remain for future research. First, the current evaluation focuses on a limited set of model scales due to computational constraints. Future work could investigate performance trends across medium and larger models (e.g., Llama-17B) to better understand the relationship between model scale and hallucination detection capabilities. Second, while this work relies on fine-tuned classifiers, it would be valuable to compare the proposed approach with zero-shot and few-shot prompting strategies using LLMs. Third, detecting missing or incomplete information remains challenging and requires further investigation, particularly for removal-type hallucinations. Finally, although synthetic hallucination injection enables controlled experimentation, validating the approach on manually annotated real-world datasets would provide stronger evidence of its applicability in practical SE workflows.

VI. THREATS TO VALIDITY

Construct Validity A primary threat involves our hallucination taxonomy (addition, removal, and reordering), which was established via an exploratory analysis of 80 reports. While this covers prominent error modes, it may not encompass all subtle or domain-specific factual inconsistencies. Furthermore, due to the lack of pre-existing human-annotated bug report hallucination corpora, we rely on an LLM-driven synthetic injection pipeline to train and evaluate our models. These synthetic perturbations serve as an operational proxy and might not perfectly mirror the semantic nuances or distribution of organic hallucinations produced in real-world software workflows. Finally, our report filtering relies on the modified PARENT_{table} metric, which emphasizes token overlap and might mischaracterize highly creative yet faithful paraphrases. To mitigate these threats, we validated our exploratory baseline with double-blind annotations achieving high agreement (Cohen’s $\kappa = 0.85$) and implemented automated validation rules during synthesis to ensure the structural and behavioral consistency of injected errors.

Internal Validity A main threat relates to using self-attention scores as an operational proxy for model interpretability; in deep learning architectures, attention weights do not always map linearly to underlying model reasoning or provide an exhaustive explanation of prediction behavior. Additionally, our text preprocessing and artifact engineering introduce configuration bias; specifically, the insertion of structural delimiters used to isolate fields (such as [S2R], [AB], and [EB]) fundamentally alters sequence lengths and input token distributions, directly influencing how attention weights are

distributed by the transformer layers. Finally, the choice of specific models (e.g., Mistral-7B for conversion and Ministral-3B for multi-task detection) introduces model-specific configuration bias. To control for these threats, we enforced a strict, uniform data-cleaning and tokenization pipeline across all configurations. Furthermore, rather than relying on a single architecture, we evaluated our section-level attention analysis across five distinct transformer models (BERT, RoBERTa, SciBERT, DeBERTa, and DistilBERT) to isolate reproducible behavioral trends rather than design-specific artifacts.

External Validity In our study, the foremost limitation lies in the structural and domain diversity of our subject systems, as our evaluation is bounded by the open-source characteristics of the BugsRepo dataset. Consequently, our findings and fine-tuned models might not immediately generalize to closed-source enterprise tracking ecosystems (e.g., Jira, ServiceNow) or highly customized, unstructured issue-tracking workflows that lack clear section boundaries. Moreover, the specific syntax of our injection vectors and the mathematical constraints of the evaluation metrics heavily shape performance outcomes, meaning model sensitivity could vary under alternative prompting styles or compression ratios. We addressed the immediate risk of overfitting and ensured statistical stability by executing our multi-task detection experiments on a sufficiently large validation partition that provides stable metric responses. To reinforce external generalizability, our future work involves expanding this benchmark to encompass heterogeneous, multi-domain software engineering datasets and validating our section-aware models in production-grade software environments.

VII. CONCLUSION

This empirical study investigated hallucinations in LLM-generated bug report summaries through a section-aware perspective using open source software repository dataset. We first conducted an exploratory analysis showing that hallucination patterns vary across different semantic sections of bug reports. Based on these observations, we constructed a synthetic benchmark by identifying less-attended bug report sections, injecting controlled hallucinations, converting structured bug reports into unstructured narratives, and filtering low-quality generations using a PARENT-based scoring strategy. We then proposed a section-aware hallucination detection approach capable of simultaneously identifying hallucinated summaries, affected report sections, and hallucination types. This empirical study results demonstrated that our proposed approach effectively supports fine-grained hallucination detection while providing concrete interpretable insights. Overall, our findings suggest that incorporating document structure into hallucination detection can improve the reliability and transparency of LLM-assisted bug report summarization systems in SE workflows. Thus, our study provides empirical evidence of hallucination risks in LLM-assisted bug-report summarization, with a detection framework for improving the reliability of software engineering artifacts.

VIII. DATA AVAILABILITY

To support the reproducibility and verifiability of our study, the datasets used and generated in this work are publicly available across two primary repositories:

- **Source Bug Reports:** The underlying structured bug reports used for our exploratory analysis and synthetic generation are sourced from the *BugsRepo* dataset, which is permanently archived and publicly accessible on Zenodo at <https://zenodo.org/records/15004067>.
- **Replication Package and Processed Data:** The complete replication package, including the controlled synthetic hallucination injection scripts, the finalized section-aware benchmark dataset, the trained model checkpoints, and evaluation scripts, is hosted anonymously to preserve the double-blind review process. It can be accessed at https://anonymous.4open.science/r/Anon_Bug_Report_Summarization-1477/data.

REFERENCES

- [1] Leveraging synthetic data for llm hallucination detection. In *SemEval*, 2024.
- [2] Shamsu Abdullahi et al. The rise of hallucination in large language models: systematic reviews, performance analysis and challenges. *Cluster Computing*, 29(2):124, Feb 2026. doi:10.1007/s10586-025-05891-z.
- [3] Jagrit Acharya and Gouri Ginde. Bugsrepo: A comprehensive curated dataset of bug reports, comments and contributors information from bugzilla. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*, EASE '25, page 986–993, New York, NY, USA, 2025. Association for Computing Machinery. doi:10.1145/3756681.3756994.
- [4] Jagrit Acharya and Gouri Ginde. Can we enhance bug report quality using llms?: An empirical study of llm-based bug report generation. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*, EASE '25, page 994–1003, New York, NY, USA, 2025. Association for Computing Machinery. doi:10.1145/3756681.3756995.
- [5] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960. arXiv:<https://doi.org/10.1177/001316446002000104>, doi:10.1177/001316446002000104.
- [6] Bhuwan Dhingra et al. Handling divergent reference texts when evaluating table-to-text generation. In Anna Korhonen, David Traum, and Lluís Màrquez, editors, *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4884–4895, Florence, Italy, July 2019. Association for Computational Linguistics. URL: <https://aclanthology.org/P19-1483/>, doi:10.18653/v1/P19-1483.
- [7] Angela Fan et al. Large language models for software engineering: Survey and open problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, pages 31–53, 2023. doi:10.1109/ICSE-FoSE59343.2023.00008.
- [8] Sen Fang et al. Representthemall: A universal learning representation of bug reports. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 602–614, 2023. doi:10.1109/ICSE48619.2023.00060.
- [9] Cuiyun Gao, David Lo, et al. A systematic literature review of code hallucinations in llms: Characterization, mitigation methods, challenges, and future directions for reliable ai, 2025. URL: <https://arxiv.org/abs/2511.00776>, arXiv:2511.00776.
- [10] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, et al. A survey on llm-as-a-judge. *The Innovation*, 2024.
- [11] Maggie Hamill and Katerina Goseva-Popstojanova. Analyzing and predicting effort associated with finding and fixing software faults. *Information and Software Technology*, 87:1–18, 2017. URL: <https://www.sciencedirect.com/science/article/pii/S0950584917300290>, doi:10.1016/j.infsof.2017.01.002.

- [12] Waka Ito, Yui Obara, Miyu Sato, and Kimio Kuramitsu. Hallucination detection on code generation with selfcheckgpt. *Journal of Information Processing*, 33:487–493, 2025. doi:10.2197/ipsjip.33.487.
- [13] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Comput. Surv.*, 55(12), March 2023. doi:10.1145/3571730.
- [14] He Jiang, Najam Nazar, et al. Prst: A pagerank-based summarization technique for summarizing bug reports with duplicates. *International Journal of Software Engineering and Knowledge Engineering*, 27(06):869–896, 2017. arXiv:https://doi.org/10.1142/S0218194017500322, doi:10.1142/S0218194017500322.
- [15] Magne Jorgensen and Martin Shepperd. A systematic review of software development cost estimation studies. *IEEE Transactions on Software Engineering*, 33(1):33–53, 2007. doi:10.1109/TSE.2007.256943.
- [16] Sahil Kale and Antonio Luca Alfeo. Lie to me: Knowledge graphs for robust hallucination self-detection in llms, 2025. URL: https://arxiv.org/abs/2512.23547, arXiv:2512.23547.
- [17] Herb Krasner. The cost of poor software quality in the us: A 2022 report. Technical report, Consortium for Information & Software Quality, 2022. URL: https://www.it-cisq.org/the-cost-of-poor-quality-software-in-the-us-a-2022-report/.
- [18] Arjun Krishna, Erick Galinkin, Leon Derczynski, and Jeffrey Martin. Importing phantoms: Measuring llm package hallucination vulnerabilities, 01 2025. doi:10.48550/arXiv.2501.19012.
- [19] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain, July 2004. Association for Computational Linguistics. URL: https://aclanthology.org/W04-1013/.
- [20] Chunhua Liu et al. Hallucinations in code change to natural language generation: Prevalence and evaluation of detection metrics. In *Proceedings of the 14th International Joint Conference on Natural Language Processing*, pages 2538–2560, Mumbai, India, December 2025. The Asian Federation of Natural Language Processing and The Association for Computational Linguistics. URL: https://aclanthology.org/2025.ijcnlp-long.137/, doi:10.18653/v1/2025.ijcnlp-long.137.
- [21] Fang Liu, Yang Liu, Lin Shi, Zhen Yang, Li Zhang, Xiaoli Lian, Zhongqi Li, and Yuchi Ma. Beyond functional correctness: Exploring hallucinations in llm-generated code. *IEEE Transactions on Software Engineering*, 52:1037–1055, 2024. doi:10.1109/tse.2026.3657432.
- [22] Rafael Lotufo, Zeeshan Malik, and Krzysztof Czarnecki. Modelling the ‘hurried’ bug report reading process to summarize bug reports. *Empirical Software Engineering*, 20(2):516–548, Apr 2015. doi:10.1007/s10664-014-9311-2.
- [23] Kishan Maharaj, , Pushpak Bhattacharyya, et al. ETF: An entity tracing framework for hallucination detection in code summaries. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 30639–30652, Vienna, Austria, July 2025. Association for Computational Linguistics. URL: https://aclanthology.org/2025.acl-long.1480/, doi:10.18653/v1/2025.acl-long.1480.
- [24] Senthil Mani, Rose Catherine, Vibha Singhal Sinha, and Avinava Dubey. Ausum: approach for unsupervised bug report summarization. In *Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering, FSE ’12*, New York, NY, USA, 2012. Association for Computing Machinery. doi:10.1145/2393596.2393607.
- [25] Joshua Maynez et al. On faithfulness and factuality in abstractive summarization. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault, editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 1906–1919, Online, July 2020. Association for Computational Linguistics. URL: https://aclanthology.org/2020.acl-main.173/, doi:10.18653/v1/2020.acl-main.173.
- [26] Diyana Muhammed et al. Selfcheck-eval: A multi-module framework for zero-resource hallucination detection in large language models. 2025. URL: https://api.semanticscholar.org/CorpusID:276107224.
- [27] Samal Mukhtar, Seonah Lee, and Jueun Heo. A multidocument summarization technique for informative bug summaries. *IEEE Access*, 12:158908–158926, 2024. doi:10.1109/ACCESS.2024.3487443.
- [28] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics, ACL ’02*, page 311–318, USA, 2002. Association for Computational Linguistics. doi:10.3115/1073083.1073135.
- [29] Bibek Paudel, Alexander Lyzhov, Preetam Joshi, and Puneet Anand. Hallucinet: Hallucination detection through context and common knowledge verification. *ArXiv*, abs/2504.07069, 2025. URL: https://api.semanticscholar.org/CorpusID:277634150.
- [30] Yunna Shao and Bangmeng Xiang. Enhancing bug report summaries through knowledge-specific and contrastive learning pre-training. *IEEE Access*, 12:37653–37662, 2024. doi:10.1109/ACCESS.2024.3368915.
- [31] Salma Begum Tamanna, Gias Uddin, et al. *ChatGPT Inaccuracy Mitigation during Technical Report Understanding: Are We There Yet?*, page 2290–2302. IEEE Press, 2025. URL: https://doi.org/10.1109/ICSE55347.2025.00145.
- [32] Chakkrit Kla Tantithamthavorn et al. Hallujudge: A reference-free hallucination detection for context misalignment in code review automation. *ArXiv*, abs/2601.19072, 2026. URL: https://api.semanticscholar.org/CorpusID:285070919.
- [33] Marcos Tileria, Santanu Kumar Dash, Profir-Petru Pârțachi, and Earl T. Barr. Hallucination inspector: A fact-checking judge for api migration, 2026. URL: https://arxiv.org/abs/2604.20202, arXiv:2604.20202.
- [34] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. URL: https://arxiv.org/abs/1706.03762, arXiv:1706.03762.
- [35] Qianqian Wang, Chris Parnin, and Alessandro Orso. Evaluating the usefulness of ir-based fault localization techniques. In *Proceedings of the 2015 international symposium on software testing and analysis*, pages 1–11, 2015.
- [36] Bangmeng Xiang and Yunna Shao. Sumllama: Efficient contrastive representations and fine-tuned adapters for bug report summarization. *IEEE Access*, 12:78562–78571, 2024. doi:10.1109/ACCESS.2024.3397326.
- [37] Yayun Zhang, Yuying Li, Mingying Fang, Xing Yuan, and Junwei Du. Brmds: an llm-based multi-dimensional summary generation approach for bug reports. *Automated Software Engg.*, 33(1), September 2025. doi:10.1007/s10515-025-00553-1.
- [38] Ziyao Zhang, Chong Wang, Yanlin Wang, Ensheng Shi, Yuchi Ma, Wanjun Zhong, Jiachi Chen, Mingzhi Mao, and Zibin Zheng. Llm hallucinations in practical code generation: Phenomena, mechanism, and mitigation. *Proceedings of the ACM on Software Engineering*, 2:481 – 503, 2024. doi:10.1145/3728894.